

Living Map Installation and Operation Guide

From an empty computer to a complete mission:
every command, in order

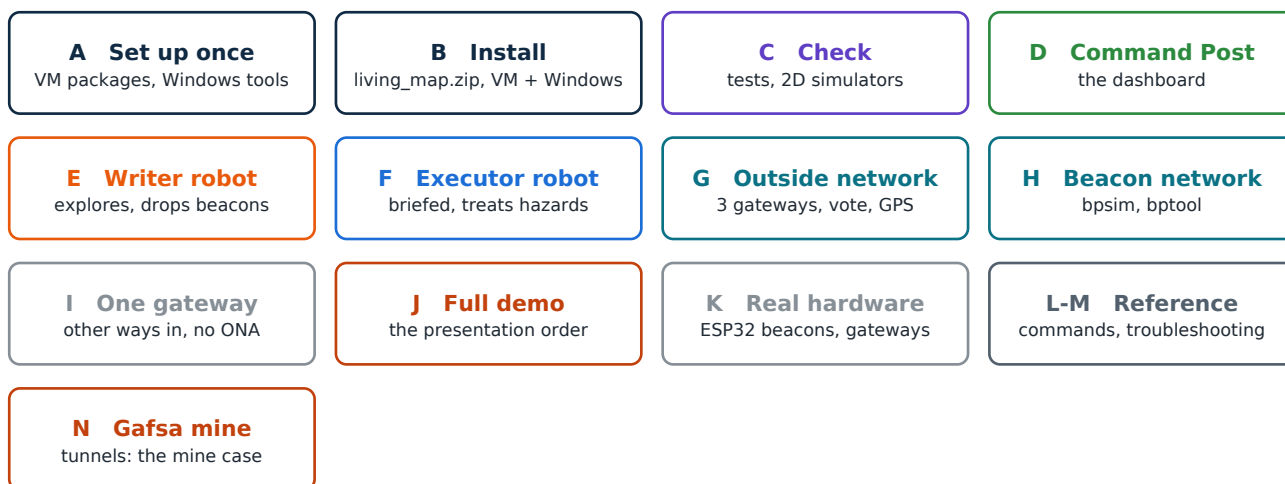
Contents

How to use this guide	4
What you deliver and install: living_map.zip	4
Where each command runs	5
Rules that avoid most problems	5
A. One-time setup	6
A.1 Ubuntu VM	6
A.2 Windows	6
A.3 Make the VM lighter and easier (optional)	6
B. Install the project	7
B.1 In the VM	7
B.2 On Windows	7
B.3 Updating later	8
C. Check that everything works (no Gazebo)	8
C.1 On Windows (double-click)	8
C.2 The same in the VM	8
C.3 Options of the 2D simulators	9
D. The Command Post dashboard	10
D.1 Start it	10
D.2 Where the dashboard is, from each place	11
D.3 Using it	11
D.4 Check the server from a terminal	11
E. The Writer robot	12
E.1 Start it	12
E.2 Watch what it does (new terminals)	12
E.3 Stop it and keep the mission	13
E.4 Change a parameter	14
E.5 Start the Writer by hand, one terminal per piece (advanced)	14
E.6 Drive by hand and test the trapdoor (optional)	15
F. The Executor robot	15
F.1 Start it	15
F.2 Watch it (new terminals)	16
F.3 Brief it from the Command Post (option ona)	16
F.4 Change what it does	17
F.5 Beacon-graph navigation (Dijkstra / A*): ready for later	18
F.6 Start the Executor by hand (advanced)	18
G. The Outside Network Area (three gateways)	19

G.1 The zone simulator: a whole mission, three gateways	19
G.2 The beacon network simulator through the vote	19
G.3 With the Gazebo robots	20
G.4 Look inside	20
H. The beacon network (LMB2)	21
H.1 Run the network simulator	21
H.2 Tests and stress matrix	22
H.3 Work with single frames (bptool)	22
I. One gateway, no ONA: other ways into the dashboard	23
I.1 Simulated network to dashboard	23
I.2 The Writer (Gazebo) to dashboard, LMB2 records	23
I.3 The Writer to dashboard, original contract (ros_bridge.py)	24
I.4 A real gateway board to dashboard	24

How to use this guide

This guide takes you from an empty computer to a complete Living Map mission: the **Writer** robot explores a contaminated building and drops LoRa beacons, the beacon network passes the records around, **three gateways** and the **Outside Network Area (ONA)** check them, locate the robots and forward everything to the **Command Post** dashboard, and the **Executor** robot, briefed from the dashboard, follows the beacons to treat the hazards. Every command you can use is here, in the order you need it, with what it does and where to type it. The companion document *2_Living_Map_Architecture.pdf* explains how each piece works inside.



A and B once, in this order. C to I and N in any order. J is the order for a presentation.

The parts of this guide.

What you deliver and install: living_map.zip

One zip holds the whole project. Unzip it in the Ubuntu VM (`~/living_map`) and on Windows (`C:\tsyp\living_map`): the same folder works in both places.

in living_map/	what it is
START_HERE.md	the first page to read: the folder, the first commands
COMMANDS.txt	every command of this guide, one per line, same sections: copy commands from there
docs/1_Living_Map_Guide.pdf	this guide: install and run everything
docs/2_Living_Map_Architecture.pdf	how everything works
docs/3_Gafsa_Mine_Case.pdf	the same system in a mine (Gafsa): dangers, sensors, robots, radio, dashboard
writer_robot/	the ROS 2 package of both robots (Writer and Executor), Gazebo worlds, 2D simulators and tests, <code>install.sh</code> , the run scripts, Windows <code>.bat</code> launchers
beacon_net/	LMB2: the beacon protocol in C, the ESP32 firmware (beacon and gateway), the network simulator <code>bpsim</code> , <code>bptool</code> , the Python mirror and the one-gateway bridge, Windows <code>.exe</code> and <code>.bat</code>
ona/	the Outside Network Area: three gateways, the vote, the robots located, frame translation, LTE / satellite queue, briefings; the zone simulator and its tests; the GALERIA originals
command_post/	the Command Post dashboard (Node.js) and its scripted demo

Where each command runs

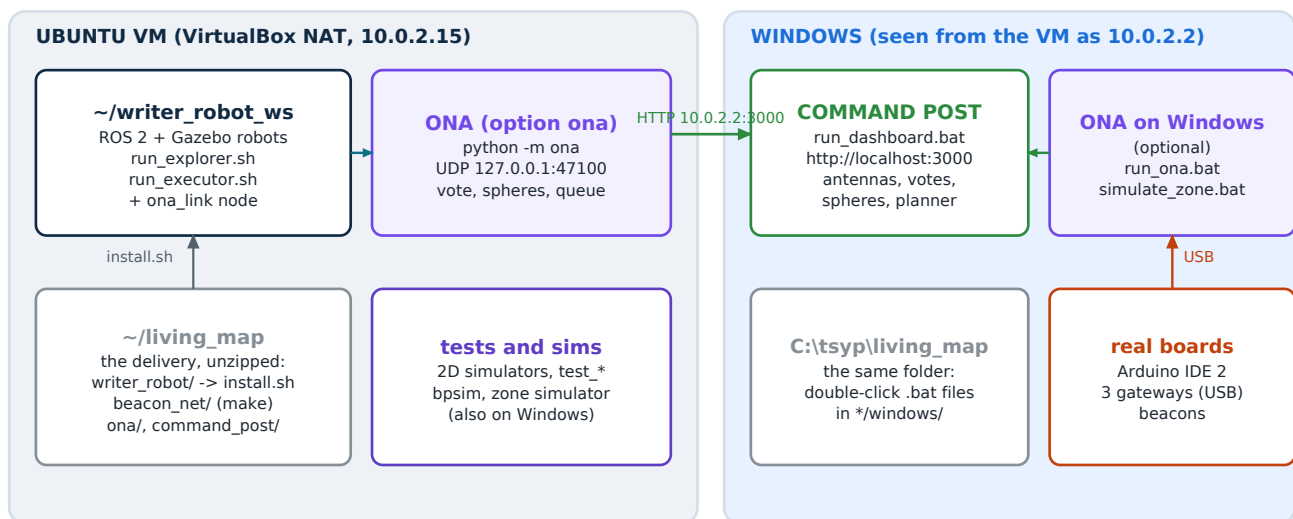
Every command has a coloured tag that tells you where to type it:

tag	meaning
VM terminal	a terminal in the Ubuntu VM (Ctrl+Alt+T opens one). ROS is loaded by your <code>~/ .bashrc</code> (A.1, B.1) and by the scripts.
Windows	a double-click on a <code>.bat</code> file, or a line in a Command Prompt (<code>cmd</code>) window
Browser	a web browser (on Windows unless it says otherwise)
Serial / board	the Serial Monitor of the Arduino IDE (115200 baud), talking to an ESP32 board
VM or Windows	works the same in both (on Windows type <code>py -3</code> instead of <code>python3</code>)

Rules that avoid most problems

- **Every command is one line.** Paste one line at a time and press Enter. Long commands wrap over two or three lines on these pages but are still one line: copy them from `COMMANDS.txt` (open it in gedit in the VM or in Notepad on Windows), because copying from a PDF can cut them into pieces.
- `~` is your home folder in the VM (`/home/<you>`). Paths starting with `C:\` are on Windows.
- Ctrl-C stops the program running in a terminal. The robot scripts stop everything they started when you press Ctrl-C.
- Anything that keeps running (a robot, the dashboard, a log you follow) needs its own terminal or window. The guide says "new terminal" when you need one.
- When something looks wrong, look at the log first. Part M lists what each log tells you.

Copy `living_map.zip` into the VM (`~/Downloads` or a shared folder) and unzip it on Windows too.



Where everything runs: the robots in the VM, everything else in the VM or on Windows. The VM reaches Windows at 10.0.2.2.

A. One-time setup

A.1 Ubuntu VM

ROS 2 Lyrical and Gazebo Jetty are already installed in the VM. These lines add everything the Living Map needs on top; running them again is harmless.

VM terminal	<code>sudo apt update</code> refresh the package lists
VM terminal	<code>sudo apt install -y ros-lyrical-ros-gz ros-lyrical-robot-localization ros-lyrical-slam-toolbox ros-lyrical-xacro ros-lyrical-robot-state-publisher ros-lyrical-joint-state-publisher ros-lyrical-cv-bridge python3-opencv ros-lyrical-teleop-twist-keyboard ros-lyrical-tf2-tools</code> the ROS packages the robots use: Gazebo bridge, EKF, SLAM, URDF tools, camera conversion, keyboard driving, TF tools
VM terminal	<code>sudo apt install -y build-essential unzip curl nodejs npm python3-numpy python3-serial python3-matplotlib</code> C compiler (beacon simulator), unzip, curl, Node.js (dashboard), numpy (ONA, simulators), serial ports, plots
VM terminal	<code>grep -q 'opt/ros/lyrical' ~/.bashrc echo 'source /opt/ros/lyrical/setup.bash' >> ~/.bashrc</code> every new terminal loads ROS 2 (adds the line only if it is missing)
VM terminal	<code>sudo usermod -aG dialout,vboxsf \$USER</code> lets you use USB serial ports and VirtualBox shared folders; takes effect after a reboot

A.2 Windows

install	why	where from
Node.js LTS	the Command Post dashboard	nodejs.org
Python 3 (tick Add python.exe to PATH)	ONA, gateway bridge, 2D simulators, tests	python.org/downloads
Arduino IDE 2 + USB driver (CP210x or CH340)	only for real ESP32 boards (part K)	arduino.cc
VS Code (optional)	reading and editing the code	code.visualstudio.com

The Python packages are installed by two .bat files once the folder is unzipped (B.2).

A.3 Make the VM lighter and easier (optional)

- VM powered off → Settings → System: about half your CPU cores (4 or more) and 6–8 GB RAM. Gazebo is CPU-bound.
- Share a Windows folder instead of copying through Downloads: Devices → Shared Folders, add `C:\tsyp`, tick Auto-mount. After the `usermod` line above and a reboot it appears at `/media/sf_tsyp`. Keep the ROS workspace itself inside the VM (builds on a shared folder are slow).
- Run the robots with headless (no Gazebo window): the single biggest speed-up. RViz still shows everything.

B. Install the project

B.1 In the VM

Copy `living_map.zip` into `~/Downloads` in the VM (drag and drop, browser download, or from the shared folder with the first line below), then run the lines in order.

VM terminal	<code>cp /media/sf_tsyp/living_map.zip ~/Downloads/</code> only if you use the shared folder of A.3
VM terminal	<code>cd ~ && rm -rf ~/living_map && unzip -o ~/Downloads/living_map.zip</code> creates <code>~/living_map/</code> with <code>writer_robot/</code> , <code>beacon_net/</code> , <code>ona/</code> , <code>command_post/</code> , <code>docs/</code> . A previous <code>~/living_map</code> is replaced (keep your own files elsewhere).
VM terminal	<code>bash ~/living_map/writer_robot/install.sh</code> copies the robots package (and a copy of <code>ona/</code>) to <code>~/writer_robot_ws/src/writer_robot</code> , puts the run scripts in <code>~/writer_robot_ws</code> , and builds. It must end with Summary: 1 package finished and the "Installed" banner. A package already there is moved to <code>~/writer_robot_backups/</code> first.
VM terminal	<code>grep -q 'writer_robot_ws/install' ~/.bashrc echo 'source</code> <code>~/writer_robot_ws/install/setup.bash' >> ~/.bashrc</code> every new terminal also loads the robots package (only if the line is missing)
VM terminal	<code>make -C ~/living_map/beacon_net/host</code> builds the beacon network simulator <code>bpsim</code> and the frame tool <code>bptool</code>
VM terminal	<code>cd ~/living_map/command_post && npm install</code> only if you run the dashboard in the VM: downloads its Node packages (once, needs internet)
VM terminal	<code>python3 -c "import numpy, serial; print('numpy', numpy.__version__, 'and pyserial</code> <code>OK')"</code> the ONA needs <code>numpy</code> ; <code>pyserial</code> only for real boards. If it fails: the second <code>apt install</code> line of A.1.

Then **close the terminal and open a new one** (it loads the package). What the installer puts in `~/writer_robot_ws`:

file or folder	use
<code>run_explorer.sh</code>	the Writer: explores, drops beacons, records the mission (part E)
<code>run_executor.sh</code>	the Executor: follows the beacons, briefed from the Command Post (part F)
<code>stop_demo.sh</code>	kills every simulation process (Gazebo, SLAM, robots, ONA, RViz)
<code>check_sim.sh</code>	health check of a running simulation (topics, transforms, Gazebo speed)
<code>missions/</code>	where the Writer saves its beacons (<code>latest.json</code> and dated copies)
<code>src/writer_robot/</code>	the package, with <code>ona/</code> inside (used by the <code>ona</code> option); the tools run from here

B.2 On Windows

Windows	Unzip <code>living_map.zip</code> into <code>C:\tsyp\</code> (right-click → Extract All → <code>C:\tsyp</code>) you get <code>C:\tsyp\living_map\</code> (a short path without spaces avoids trouble)
Windows	<code>C:\tsyp\living_map\writer_robot\windows\install_python_deps.bat</code> once: <code>numpy</code> , <code>matplotlib</code> and <code>OpenCV</code> for the 2D simulators and tests
Windows	<code>C:\tsyp\living_map\ona\windows\install_deps.bat</code> once: <code>numpy</code> and <code>pyserial</code> for the ONA

- The first time you run `bpsim.exe` or `bptool.exe`, Windows SmartScreen may warn (unsigned program):
More info → Run anyway.

- The first `run_dashboard.bat` installs the dashboard packages by itself (needs internet once).
- When Windows asks whether Node.js or Python may use the network, allow **private networks**: otherwise the VM cannot reach the dashboard or the ONA.

B.3 Updating later

A new `living_map.zip`: run the B.1 lines again (`unzip`, `install.sh`, `make`) and unzip it again on Windows. `install.sh` keeps every previous package and script in `~/writer_robot_backups/` and never deletes a mission. After changing a file in `~/writer_robot_ws/src/writer_robot` yourself, rebuild with the line below.

VM terminal `cd ~/writer_robot_ws && colcon build --packages-select writer_robot --symlink-install`
rebuild the package by hand, then open a new terminal

C. Check that everything works (no Gazebo)

These run in seconds to minutes and prove that the robot logic, the ONA and the protocol work before you start the heavy simulation. They are also the fastest way to show a whole mission on a laptop without the VM.

C.1 On Windows (double-click)

Windows `C:\tsyp\living_map\writer_robot\windows\sim_mission_big_map.bat`
the whole mission in the 2D simulator on the big map (Writer, then Executor), then opens a picture of both runs

Windows `C:\tsyp\living_map\writer_robot\windows\sim_mission_retreat_test.bat`
the same on the small map

Windows `C:\tsyp\living_map\writer_robot\windows\sim_big_map.bat`
only the Writer exploring the big map

Windows `C:\tsyp\living_map\writer_robot\windows\sim_retreat_test.bat`
the Writer turning back at the purple block

Windows `C:\tsyp\living_map\writer_robot\windows\run_all_tests.bat`
every robot test: 21 Writer runs, 26 missions (the Gafsa mine included), 70 node checks, 21 beacon-graph checks, 18 robot + ONA checks (15–20 minutes)

Windows `C:\tsyp\living_map\ona\windows\run_tests.bat`
the ONA: 70, 20 and 3 checks (about 2 minutes)

Windows `C:\tsyp\living_map\beacon_net\windows\run_tests.bat`
`C` ⇏ Python protocol cross-checks

Windows `C:\tsyp\living_map\beacon_net\windows\run_simulation.bat`
one beacon-network simulation with its 13 PASS/FAIL checks

C.2 The same in the VM

VM terminal `cd ~/writer_robot_ws/src/writer_robot`
go to the package: the robot tools run from here

VM terminal `python3 tools/sim_mission.py --png /tmp/mission.png && xdg-open /tmp/mission.png`
whole mission on the big map, then shows the picture

VM terminal	<code>python3 tools/sim_mission.py --world worlds/retreat_test.world --png /tmp/mission_small.png</code> whole mission on the small map
VM terminal	<code>python3 tools/sim2d.py --png /tmp/writer.png</code> the Writer only, big map, with checks
VM terminal	<code>python3 tools/run_tests.py</code> 21 Writer runs (4 maps incl. the Gafsa mine, several seeds), PASS/FAIL per run
VM terminal	<code>python3 tools/run_mission_tests.py</code> 26 whole missions, including 4 Command Post briefings and 4 in the Gafsa mine (about 6 minutes)
VM terminal	<code>python3 tools/test_nodes.py</code> the real ROS node files on a small ROS stand-in: 70 checks
VM terminal	<code>python3 tools/test_beacon_graph.py</code> beacon-graph routes (Dijkstra / A*): 21 checks
VM terminal	<code>python3 tools/test_ona_link.py</code> the robots and the real ONA together, no ROS needed: 18 checks (about 2 minutes)
VM terminal	<code>cd ~/living_map/ona && python3 tests/test_ona.py && python3 tests/test_zone.py && python3 tests/test_vs_beaconnet.py</code> the ONA: 70, 20 and 3 checks (about 2 minutes)
VM terminal	<code>make -C ~/living_map/beacon_net/host test</code> protocol cross-checks + firmware compiled for the PC + one network simulation
test	a good run ends with
<code>run_tests.py</code>	passed 21 / 21
<code>run_mission_tests.py</code>	passed 26 / 26
<code>test_nodes.py</code>	70 / 70 checks passed
<code>test_beacon_graph.py</code>	21 / 21 checks passed
<code>test_ona_link.py</code>	18 / 18 checks passed: ALL PASSED
ONA tests	70 / 70, 20 / 20 and 3 / 3 checks passed
<code>make test</code>	every cross-check OK and the simulation's 13 checks PASS

C.3 Options of the 2D simulators

tool	option	meaning
sim2d.py	<code>--world FILE</code>	world to use (default the big map)
	<code>--minutes M</code>	simulated time limit
	<code>--seed N</code>	random seed (sensor noise, start jitter)
	<code>--start X Y</code>	start position
	<code>--png FILE</code>	draw the run into a picture
	<code>--quiet / --verbose / --trace</code>	less output / every planning decision / every step
	<code>--min-area PX</code>	camera blob threshold, like <code>min_area_px</code>
	<code>--expect-turn-back</code>	fail unless the robot turns back at a hazard
sim_mission.py	<code>--self-hits</code>	add fake LiDAR hits on the robot body (robustness test)
	<code>--world, --seed, --min-area, --png, --quiet, --verbose</code>	as above

tool	option	meaning
	--mission FILE	skip the Writer: use a recorded mission file (e.g. from Gazebo)
	--save-mission FILE	save the Writer's mission file
	--frame-error M	shift the Executor's map against the Writer's (misalignment test)
	--radio-offset N	the radio starts in the middle of the record cycle
	--briefing T:ID,ID,...[:stay]	a Command Post briefing at simulated second T (targets in order; stay = do not return to the exit); T:abort calls the Executor back
	--wait-briefing S	the Executor waits S seconds at the exit for a briefing
VM terminal	python3 tools/sim_mission.py --mission ~/writer_robot_ws/missions/latest.json --png /tmp/replay.png replay a mission recorded in Gazebo in the 2D Executor, in seconds	
VM terminal	python3 tools/sim_mission.py --mission missions/demo_big.json --wait-briefing 30 --briefing 5:25,11,20 a briefing: the Executor treats #25, #11, #20 in that order, then RESULT: ALL CHECKS PASSED	
VM terminal	python3 tools/sim_mission.py --mission missions/demo_big.json --briefing 40:20,30:stay re-targeted during the mission; it stays at its last target	
VM terminal	python3 tools/sim_mission.py --mission missions/demo_big.json --briefing 45:abort called back: it stops and follows the beacons out	

D. The Command Post dashboard

D.1 Start it

Windows	C:\tsyp\living_map\beacon_net\windows\run_dashboard.bat recommended: the dashboard on Windows (installs its packages the first time). Keep the window open.
Browser	http://localhost:3000 open the dashboard in the Windows browser
Windows	C:\tsyp\living_map\beacon_net\windows\run_dashboard_demo.bat dashboard + a scripted mission with no robot and no simulator: votes, both robots on their circles, a lying gateway, a SLAM drift, the satellite link
VM terminal	cd ~/living_map/command_post && npm start alternative: the dashboard inside the VM
VM terminal	firefox http://localhost:3000 & open it in the VM
VM terminal	cd ~/living_map/command_post && npm run demo VM: dashboard + scripted mission
VM or Windows	npm run mock (in command_post/) only the scripted feeder, against a dashboard already running
VM or Windows	ONA=0 npm run demo VM syntax: the scripted mission with one gateway and no ONA

VM or Windows	TARGET=http://192.168.1.20:3000 node mock_feeder.js VM syntax: scripted feeder against another machine (Windows: set TARGET=... first)
VM or Windows	PORT=4000 GATEWAYS=gw1,gw2,gw3 npm start VM syntax: another port and the list of expected gateways

D.2 Where the dashboard is, from each place

dashboard runs on	from the Windows browser	from programs in the VM	from another laptop
Windows	http://localhost:3000	http://10.0.2.2:3000	http://<Windows LAN IP>:3000
the VM	not reachable with plain NAT (add a VirtualBox port forward 3000 → 3000)	http://localhost:3000	through the same port forward

D.3 Using it

- **Pins** are beacons, coloured by kind; they fade as each record's confidence decays. Dashed arrows are the next links (the way out). Click a pin: severity, confidence now and at creation, age, accuracy, hops, signal, its vote, and its whole route to the exit in green with the position-error circle.
- **Votes**: ✓3/3 or ✓2/3 = confirmed by two or three gateways. A dotted, faded pin marked 1/3 was reported by one gateway only: it is not counted and never replaces a confirmed version.
- **Lost beacons**: red dashed ring and "LOST?". Hollow pins: aged-out events kept as route markers. Grey: retracted.
- **Gateways and robots**: three antennas around the building; around them, each one's measured distance to a robot as a circle; the robot where they cross, inside its error ellipse; a small dashed circle where the robot thinks it is. **Spheres** toggles the circles; **Follow** keeps a robot in view.
- **Outside network area panel**: uplink (LTE / SATELLITE / NO UPLINK), each gateway's agreement (IGNORED after 3 strikes), records confirmed / waiting / conflicts, queue, frame check (holds / DRIFTED), briefing and its acknowledgement. **Executor** and **Writer** panels: measured by N gateways ± error, own SLAM agrees or DRIFT, phase, battery, mission progress.
- **Top bar**: one chip per gateway, records held, △ HMAC rejects. **Feed**: every event and ONA alert.
- **Mission planner**: *Select on map*, click 2–3 pins, reorder, *Dispatch mission* (untick "back to the exit afterwards" to keep the Executor there). Or open a pin and press *Executor path to here*. *Call the Executor back* sends a signed ABORT.
- Base maps Dark / Satellite / Street / Offline grid (automatic without internet).
- Restarting the dashboard (Ctrl-C in its window, then start it again) clears every pin: do it between a simulator demo and a robot demo so the two do not mix.

D.4 Check the server from a terminal

VM terminal	curl -s http://10.0.2.2:3000/api/health from the VM to a dashboard on Windows: {"ok":true,...} proves the network path works (use localhost if the dashboard runs in the VM)
VM terminal	curl -s http://10.0.2.2:3000/api/state head -c 600 the start of the stored state

VM terminal	<pre>curl -s -X POST http://10.0.2.2:3000/api/beacon -H "Content-Type: application/json" -d '{"beacon_id":99,"event_type":"gas","value":50,"lat":36.8066,"lon":10.1816,"ttl":20, "gateway_id":"test"}'</pre> post one test beacon: a gas pin appears at once
Windows	<pre>curl.exe -s http://localhost:3000/api/beacons</pre> Windows: the last 200 beacons as JSON
Browser	<pre>http://localhost:3000/api/mission-dispatch/latest</pre> the last mission dispatched from the planner

E. The Writer robot

E.1 Start it

VM terminal	<pre>~/writer_robot_ws/run_explorer.sh</pre> big map (default): Gazebo + robot + SLAM + vision + beacons + mission recorder + explorer + RViz
VM terminal	<pre>~/writer_robot_ws/run_explorer.sh headless</pre> the same without the Gazebo window (much faster; watch RViz)
VM terminal	<pre>~/writer_robot_ws/run_explorer.sh retreat</pre> the small map: a purple block after a corner, the robot must turn back (quick, good for a first test)
VM terminal	<pre>~/writer_robot_ws/run_explorer.sh small</pre> the original small arena
VM terminal	<pre>~/writer_robot_ws/run_explorer.sh headless ona</pre> with the Outside Network Area: every beacon reaches the dashboard through three gateways as it is dropped, and the Writer appears on its distance circles (start the dashboard first, part D)
VM terminal	<pre>~/writer_robot_ws/run_explorer.sh retreat headless ona</pre> options combine in any order
VM terminal	<pre>~/writer_robot_ws/run_explorer.sh mine headless ona</pre> the Gafsa mine (part N)
VM terminal	<pre>~/writer_robot_ws/run_explorer.sh headless gpu</pre> try the graphics card (VirtualBox 3D acceleration ON, see J.1); if Gazebo does not start, drop gpu
VM terminal	<pre>~/writer_robot_ws/run_explorer.sh headless slow</pre> slower robot (0.5 m/s, slower turns), in case it bumps into things on your computer
VM terminal	<pre>~/writer_robot_ws/run_explorer.sh ekf</pre> experiment only: odometry from the wheels + IMU (EKF) instead of Gazebo's true motion (the default)

The script prints == [0/5] cleanup == ... == [5/5] RViz ==, waits for /scan, brings SLAM up (configure, activate) and waits for the map→odom transform before the robot moves. That takes 30–90 s in a VM. Leave this terminal alone; Ctrl-C in it stops everything.

E.2 Watch what it does (new terminals)

VM terminal	<pre>tail -f /tmp/explorer.log</pre> the robot's decisions: targets, RETREAT, LOOK, recoveries, EXPLORATION COMPLETE, DONE
VM terminal	<pre>grep -E 'time so far total time recover' /tmp/explorer.log</pre> where the time goes, one line per simulated minute: driving, camera looks, planning, recovering

VM terminal	<code>bash ~/writer_robot_ws/check_sim.sh</code> health check: rate of every topic, the odometry and SLAM transforms, Gazebo speed, last log lines. Send its output if something does not work.
VM terminal	<code>ros2 topic echo /writer/explorer_status</code> the state and what it is doing (Ctrl-C to stop)
VM terminal	<code>grep health /tmp/vision.log</code> camera health every 10 s: colour frames, depth frames, hazard frames per colour (M.2)
VM terminal	<code>tail -f /tmp/vision.log</code> every VISION EVENT (a hazard that gets a beacon)
VM terminal	<code>tail -f /tmp/beacon.log</code> every beacon dropped, with its link
VM terminal	<code>tail -f /tmp/mission.log</code> every record written to the mission file (kind + frame)
VM terminal	<code>ros2 topic echo /writer/beacon_dropped</code> the beacons as JSON (id, type, x, y, link_id, hazard x/y)
VM terminal	<code>ros2 topic echo /writer/hazard</code> every camera sighting (type, map x/y, range)
VM terminal	<code>ros2 topic echo /outside_network/beacon_gps</code> the beacons in lat/lon
VM terminal	<code>tail -f /tmp/ona.log</code> option ona: the ONA (uplink, queue, each gateway, records confirmed, the robot located), a line every 5 s

A normal run reads like this in `/tmp/explorer.log` (small map):

```
map and pose received - exploring
first a full turn on the spot so SLAM maps the surroundings
new hazard: radiation at (6.75, 2.99)
RETREAT: radiation ahead (2.8 m) -> returning the way it came (1.6 m)
retreat finished - exploring the rest of the map
no reachable frontier and nothing left for the camera - EXPLORATION COMPLETE
DONE - robot stopped
total time 1.0 min: driving 27 s, camera looks 13 s, planning 5 s, retreating 6 s, going home 8 s; driven 18 m
```

In RViz the map grows; the blue line is the planned path; coloured discs are hazard keep-out zones; each beacon is a post with a ball on top (green EXIT, blue trail, magenta radiation, orange fire, yellow gas) and its id, with light-blue lines to its next beacon; the state is written above the robot.

E.3 Stop it and keep the mission

VM terminal	<code>Ctrl-C</code> (in the <code>run_explorer.sh</code> terminal) stops everything the script started, once the log says DONE
VM terminal	<code>bash ~/writer_robot_ws/stop_demo.sh</code> from any terminal: kills every simulation process (if a window hangs, or before starting again)
VM terminal	<code>ls -l ~/writer_robot_ws/missions/</code> <code>latest.json</code> (always the last run) and <code>mission_<date>.json</code> copies
VM terminal	<code>python3 -m json.tool ~/writer_robot_ws/missions/latest.json less</code> read the mission file (q to quit)

```
VM terminal  grep -c '"frame"' ~/writer_robot_ws/missions/latest.json
how many beacons were recorded
```

```
VM terminal  cp ~/writer_robot_ws/missions/latest.json ~/writer_robot_ws/missions/good_run.json
keep a good run under its own name (the next Writer run overwrites latest.json)
```

E.4 Change a parameter

Parameters are read when a node starts. To change one, stop that node and start it again with `-p`, while the rest keeps running.

```
VM terminal  pkill -f frontier_explorer; ros2 run writer_robot frontier_explorer --ros-args -p
use_sim_time:=true -p v_max:=0.3 -p verbose:=true > /tmp/explorer.log 2>&1 &
restart the explorer slower, logging every planning decision. It re-plans from where the robot is.
```

```
VM terminal  pkill -f vision_event_detector; ros2 run writer_robot vision_event_detector
--ros-args -p use_sim_time:=true -p min_area_px:=400 > /tmp/vision.log 2>&1 &
restart the camera node with a smaller blob threshold (sees blocks from farther away)
```

```
VM terminal  ros2 param list /frontier_explorer
all parameters of a running node
```

```
VM terminal  ros2 param get /frontier_explorer keepout_radius
the value a node is using
```

node	useful parameters (default)
frontier_explorer	v_max 0.70 · w_max 1.6 · look_w_max 1.6 · stuck_time 8 · keepout_radius 1.2 · coverage true · camera_range 3.2 · robot_radius 0.36 · retreat_min 1.5 · retreat_clear 2.5 · home_on_finish true · verbose false
vision_event_detector	min_area_px 800 · drop_range_m 4.0 · dedup_radius_m 1.5 · camera_height_m 0.135 · hazard_width_m 0.30
beacon_drop_node	turn_threshold_deg 35 · max_trail_gap_m 2.5 · trail_dedup_m 1.2 · pass_radius_m 0.5 · magazine_capacity 120 · exit_beacon true
mission_recorder	mission_dir · anchor_lat 36.8065 · anchor_lon 10.1815 · key demo · net_id 42
ona_link	robot writer/executor · world_file · ona_config · ona_host 127.0.0.1 · ona_port 47100 · ping_period 2.0

The full list, with every default read from the code, is in the Architecture document (sections 5.7, 6.3, 11.6).

E.5 Start the Writer by hand, one terminal per piece (advanced)

This is what `run_explorer.sh` does. Use it to restart a single piece or to see its output live. Type the lines in this order, each in a new terminal, waiting for each to be up.

```
VM terminal  bash ~/writer_robot_ws/stop_demo.sh
T0: clean start
```

```
VM terminal  ros2 launch writer_robot gazebo_sim.launch.py
T1: Gazebo, robot, bridges (incl. the odometry TF). Add headless:=true for no window,
world:=<file> for another map.
```

```
VM terminal  ros2 topic hz /scan
check: about 8 Hz. Ctrl-C, then continue.
```

```
VM terminal  ros2 run slam_toolbox async_slam_toolbox_node --ros-args --params-file $(ros2 pkg
prefix writer_robot)/share/writer_robot/config/slam_toolbox_params.yaml -p
use_sim_time:=true
T2: SLAM (inactive until the next two lines)
```

VM terminal	<code>ros2 lifecycle set /slam_toolbox configure</code> T3: then...
VM terminal	<code>ros2 lifecycle set /slam_toolbox activate</code> ...SLAM starts publishing /map and map→odom
VM terminal	<code>ros2 run writer_robot vision_event_detector --ros-args -p use_sim_time:=true</code> T3: camera
VM terminal	<code>ros2 run writer_robot beacon_drop_node --ros-args -p use_sim_time:=true</code> T4: beacons
VM terminal	<code>ros2 run writer_robot mission_recorder --ros-args -p use_sim_time:=true</code> T5: mission file
VM terminal	<code>ros2 run writer_robot frontier_explorer --ros-args -p use_sim_time:=true</code> T6: the brain (the robot starts moving)
VM terminal	<code>rviz2 -d \$(ros2 pkg prefix writer_robot)/share/writer_robot/config/writer.rviz</code> <code>--ros-args -p use_sim_time:=true</code> T7: RViz

E.6 Drive by hand and test the trapdoor (optional)

VM terminal	<code>ros2 run teleop_twist_keyboard teleop_twist_keyboard</code> drive with the keyboard (i, j, l, comma, k). Do not run it together with the explorer.
VM terminal	<code>ros2 topic pub -1 /trapdoor_cmd std_msgs/msg/Float64 "{data: -1.2}"</code> open the beacon trapdoor
VM terminal	<code>ros2 topic pub -1 /trapdoor_cmd std_msgs/msg/Float64 "{data: 0.0}"</code> close it

F. The Executor robot

F.1 Start it

The Executor needs a mission file: the one your Writer just recorded, or a ready-made one. Stop the Writer first (Ctrl-C): one robot runs at a time.

VM terminal	<code>~/writer_robot_ws/run_executor.sh</code> uses <code>~/writer_robot_ws/missions/latest.json</code> (your last Writer run). The world is rebuilt with a beacon model at every drop point.
VM terminal	<code>~/writer_robot_ws/run_executor.sh demo</code> ready-made mission on the big map (43 beacons, 4 hazards): no Writer run needed
VM terminal	<code>~/writer_robot_ws/run_executor.sh demo retreat</code> ready-made mission on the small map (about 1 minute)
VM terminal	<code>~/writer_robot_ws/run_executor.sh ~/writer_robot_ws/missions/good_run.json</code> any saved mission file
VM terminal	<code>~/writer_robot_ws/run_executor.sh demo headless</code> no Gazebo window (faster); add <code>headless</code> to any of the above
VM terminal	<code>~/writer_robot_ws/run_executor.sh demo headless ona</code> with the Outside Network Area: it waits at the exit for a briefing from the dashboard (F.3)

VM terminal `~/writer_robot_ws/run_executor.sh mine headless ona`
the ready-made Gafsa mine mission (part N)

VM terminal `~/writer_robot_ws/run_executor.sh demo headless slow`
slower robot (0.5 m/s)

VM terminal `~/writer_robot_ws/run_executor.sh demo ekf`
experiment only: wheel + IMU odometry (EKF)

F.2 Watch it (new terminals)

VM terminal `tail -f /tmp/executor.log`
the mission: records received, briefing, targets, GOTO, approaching, TREATED, RETURN, MISSION COMPLETE, and a "time so far" line every simulated minute

VM terminal `cat /tmp/exec_radio.log`
the LoRa receiver: how many frames were verified and decoded

VM terminal `grep -E "skipped|approach failed|unreachable|attempt|treating from here" /tmp/executor.log`
why a hazard was not treated at the first try (and what the robot did about it)

VM terminal `ros2 topic echo /executor/status`
phase and progress

VM terminal `ros2 topic echo /executor/mission_events`
one JSON message per treated hazard

VM terminal `ros2 topic echo /executor/lora_rx --once`
one decoded beacon record as the robot receives it

VM terminal `grep health /tmp/exec_vision.log`
the Executor's camera health

```
[executor] 5 beacon records (3 trail, 2 other), exit #0; mission: 1 target(s) RADIATION #4
[executor] GOTO RADIATION #4: 5 beacons, 6.0 m along the beacon chain
[executor] at beacon #4: approaching the radiation to 1.6 m
[executor] camera confirms RADIATION #4 (0.02 m from its record)
[executor] RADIATION #4 TREATED (placing radiation shielding, from 1.90 m, confirmed by camera)
[executor] RETURN to the exit: following 5 "next" hops
[executor] MISSION COMPLETE: 1/1 targets treated, back at the exit
```

(demo `retreat`; on the big map there are 43 records and 4 targets, each with its own GOTO ... TREATED.) In RViz: the beacons and their links; a green line through the beacons still to visit; an orange line for the current leg; each hazard zone turns green when treated. In Gazebo a green disc appears around every treated hazard.

F.3 Brief it from the Command Post (option `ona`)

With `ona`, the Executor waits 60 s of simulated time at the exit for a briefing. Start the dashboard first (part D).

VM terminal `~/writer_robot_ws/run_executor.sh headless ona`
the mission the Writer recorded (`run_executor.sh demo headless ona` for the ready-made one)

Browser (dashboard) Select on map → click 2–3 hazards → Dispatch mission
the ONA signs it, the gateways transmit it, the robot checks it: "✓ acknowledged" in the ONA panel within a few seconds; it treats them in your order. Untick "back to the exit afterwards" to keep it there.

VM terminal	<code>grep -iE "briefing TREATED COMPLETE" /tmp/executor.log</code> what the Executor did with the briefing
VM terminal	<code>ros2 topic echo /executor/briefing</code> the briefing as the robot received it (latched)
VM terminal	<code>ros2 topic echo /executor/telemetry</code> phase, mission id, acknowledgement, done / total: what goes into its position reports
VM terminal	<code>ros2 topic echo /ona_link/status</code> the robot's radio: pings sent, records, briefings, simulation speed
Browser	(dashboard) Call the Executor back a signed ABORT: it stops and follows the beacons out
VM terminal	<code>WAIT_BRIEFING=180 ~/writer_robot_ws/run_executor.sh demo headless ona</code> more time to dispatch (simulated seconds; Gazebo in a VM is slower than real time). A briefing that arrives later still re-targets it.

F.4 Change what it does

VM terminal	<code>kill -f executor_node; ros2 run writer_robot executor_node --ros-args -p use_sim_time:=true -p target_kinds:=RADIATION,GAS -p treat_time:=10.0 > /tmp/executor.log 2>&1 &</code> restart only the Executor brain: treat only radiation and gas, 10 s each (it rebuilds the tree from the radio in a few seconds)
VM terminal	<code>kill -f executor_node; ros2 run writer_robot executor_node --ros-args -p use_sim_time:=true -p target_ids:='"13,30"' > /tmp/executor.log 2>&1 &</code> only beacons 13 and 30

parameter	default	meaning
target_kinds	RADIATION,THERMAL,GAS, VICTIM	what to treat (comma-separated, no spaces on the command line)
target_ids	(all)	only these beacon ids
wait_briefing	0 (60 with ona)	seconds to wait at the exit for a Command Post briefing
standoff / victim_standoff	1.6 / 0.8	distance from the hazard while treating (m)
treat_time	4.0	seconds per target
treat_max	3.0	a hazard in plain sight this close can be treated from where the robot is (m)
retry_skipped	true	one more attempt at every skipped hazard before going home
min_confidence	0.15	skip hazards that have faded below this (a briefed hazard is always taken)
age_mode	mission	wallclock = age records by the real clock (gas fades in minutes)
keepout_radius, v_max	1.2, 0.70	as for the Writer (slow: 0.5 m/s)
beacon_radio_sim: mission_file, key, rate_hz	latest.json, demo, 20	the receiver

F.5 Beacon-graph navigation (Dijkstra / A*): ready for later

`writer_robot/beacon_graph_nav.py` treats the beacons as the points of a graph (tree links + wall-free shortcuts) and finds routes with Dijkstra or A*. The Executor does not use it yet; the end of the file lists the five steps to plug it in. Try it without ROS (in `~/writer_robot_ws/src/writer_robot`):

VM or Windows	<code>python3 tools/beacon_graph_demo.py --png /tmp/graph.png</code> routes from the EXIT to every hazard: tree, Dijkstra and A*, plus a picture
VM or Windows	<code>python3 tools/beacon_graph_demo.py --mission ~/writer_robot_ws/missions/latest.json --png /tmp/mine.png</code> the same on the mission your Writer just recorded in Gazebo
VM or Windows	<code>python3 tools/beacon_graph_demo.py --link-radius 0</code> tree only (no shortcuts), for comparison
Windows	<code>C:\tsyp\living_map\writer_robot\windows\beacon_graph_demo.bat</code> Windows: the demo with its picture

F.6 Start the Executor by hand (advanced)

VM terminal	<code>ros2 run writer_robot mission_world --mission ~/writer_robot_ws/missions/latest.json --out /tmp/executor_world.world</code> T1: build the world with the beacons in place
VM terminal	<code>ros2 launch writer_robot gazebo_sim.launch.py world:=/tmp/executor_world.world robot:=executor</code> T1: Gazebo with the blue Executor
VM terminal	<code>ros2 run slam_toolbox async_slam_toolbox_node --ros-args --params-file \$(ros2 pkg prefix writer_robot)/share/writer_robot/config/slam_toolbox_params.yaml -p use_sim_time:=true</code> T2: SLAM; then <code>ros2 lifecycle set /slam_toolbox configure</code> and ... activate in T3
VM terminal	<code>ros2 run writer_robot beacon_radio_sim --ros-args -p use_sim_time:=true -p mission_file:=\$HOME/writer_robot_ws/missions/latest.json</code> T3: the LoRa receiver
VM terminal	<code>ros2 run writer_robot vision_event_detector --ros-args -p use_sim_time:=true -r /writer/hazard:=/executor/hazard -r /writer/events:=/executor/vision_events -r /writer/vision_target:=/executor/vision_target</code> T4: the Executor's camera (topics renamed)
VM terminal	<code>ros2 run writer_robot executor_node --ros-args -p use_sim_time:=true</code> T5: the brain
VM terminal	<code>rviz2 -d \$(ros2 pkg prefix writer_robot)/share/writer_robot/config/executor.rviz --ros-args -p use_sim_time:=true</code> T6: RViz
VM terminal	<code>cd ~/writer_robot_ws/src/writer_robot/ona && python3 -m ona --config ona_config_gazebo_big.json --udp 127.0.0.1:47100 --command-post http://10.0.2.2:3000</code> T7 (optional): the ONA
VM terminal	<code>ros2 run writer_robot ona_link --ros-args -p use_sim_time:=true -p robot:=executor -p world_file:=/tmp/executor_world.world -p mission_file:=\$HOME/writer_robot_ws/missions/latest.json</code> T8 (optional): the robot's radio to the three gateways; then start T5 with <code>-p wait_briefing:=60.0</code>

G. The Outside Network Area (three gateways)

The ONA is the only bridge between the zone and the outside world. Three gateways listen around the building; the ONA accepts a record only when two of them heard the identical signed bytes, locates the robots from the gateways' distance measurements (like GPS), translates everything to GPS and forwards it to the Command Post over LTE, or over satellite when LTE fails. It also signs and sends the Command Post's briefings to the Executor. It is one program (`python -m ona`) that runs in the VM or on Windows; its inputs are real gateways on USB, robots in Gazebo, the zone simulator or the beacon simulator.

WATCH OUT Start the dashboard first (part D). Run **one ONA at a time**: two would both post to the dashboard, and the second cannot open UDP port 47100.

G.1 The zone simulator: a whole mission, three gateways

No ROS, no Gazebo: a built-in building with 16 beacons, three gateways and an Executor, as the gateways hear it. Two windows: the ONA, then the zone.

Windows	<code>C:\tsyp\living_map\ona\windows\run_ona.bat</code> window 1: the ONA, listening on UDP 47100, posting to <code>http://localhost:3000</code>
Windows	<code>C:\tsyp\living_map\ona\windows\simulate_zone.bat</code> window 2: the mission. On the dashboard: select 2-3 hazards and Dispatch mission within a minute; the Executor acknowledges and treats them in that order.
Windows	<code>C:\tsyp\living_map\ona\windows\simulate_faults.bat</code> instead of window 2: gw3 lies (IGNORED), gw2 forges a victim at 60 s (stays 1/3), beacon 6 is crushed at 90 s (LOST?), the robot's SLAM slips 2.9 m at 150 s (SLAM DRIFT)
Windows	<code>C:\tsyp\living_map\ona\windows\run_ona.bat --lte-outage 60-150</code> instead of window 1 (in cmd): LTE down from 60 s to 150 s: SATELLITE on the dashboard, only the essentials go out
VM terminal	<code>cd ~/living_map/ona && ./run_ona.sh --command-post http://10.0.2.2:3000</code> the same ONA in the VM (dashboard on Windows)
VM terminal	<code>cd ~/living_map/ona && ./simulate_zone.sh</code> second VM terminal: the nominal zone
VM terminal	<code>cd ~/living_map/ona && ./simulate_zone.sh --liar gw3 --forge gw2@60 --kill 6@90 --slam-slip 150:2.5,-1.5</code> or the zone with every fault

G.2 The beacon network simulator through the vote

Windows	<code>C:\tsyp\living_map\ona\windows\bpsim_to_ona.bat</code> the beacons' own C code, 14 beacons, three gateways, 10× real time, through the ONA to the dashboard (about 6 minutes): every record ✓3/3, beacon 6 LOST at minute 25, the attacker's frames rejected
VM terminal	<code>cd ~/living_map/ona && ../beacon_net/host/bpsim --gateways 3 --speed 20 --quiet python3 -m ona --stdin --config ona_config_bpsim.json --command-post http://10.0.2.2:3000</code> the same in the VM

G.3 With the Gazebo robots

The `ona` option of both run scripts starts, in the VM, the ONA (posting to the dashboard on Windows) and the robot's `ona_link` node, which plays the three gateways' radio. Commands are in E.1 and F.3; variants:

VM terminal	<code>COMMAND_POST=http://192.168.1.20:3000 ~/writer_robot_ws/run_executor.sh demo headless ona</code> a dashboard on another computer
Windows	<code>C:\tsyp\living_map\ona\windows\run_ona_for_gazebo.bat</code> the ONA on Windows instead of the VM (big map; add <code>--config ona_config_gazebo_retreat.json</code> for the small one). Allow Python on private networks when asked.
VM terminal	<code>ONA_HOST=10.0.2.2 ~/writer_robot_ws/run_executor.sh demo headless ona</code> then the robot sends its radio lines to the Windows ONA (works the same with <code>run_explorer.sh</code>)
VM terminal	<code>bash ~/writer_robot_ws/stop_demo.sh</code> stops everything, the ONA and <code>ona_link</code> included

G.4 Look inside

VM terminal	<code>tail -f /tmp/ona.log</code> the ONA started by the <code>ona</code> option: link, queue, each gateway, records confirmed, robots
VM terminal	<code>cat /tmp/ona_link.log</code> the robot's radio node: gateways, walls, briefings received
VM terminal	<code>python3 -c "import sqlite3; [print(r) for r in sqlite3.connect('/tmp/ona_queue.sqlite3').execute('select ts, event, detail_json from audit order by id desc limit 15')]"</code> the last 15 decisions of the ONA (rejected frames, votes, alerts, link changes, briefings)
VM or Windows	<code>python3 -m ona --help</code> (in <code>ona/</code>) every option

<code>python3 -m ona</code> option	meaning
<code>--config FILE</code>	gateways (positions), anchor, key: <code>ona_config.json</code> (zone simulator), <code>ona_config_gazebo_big.json</code> / <code>_retreat.json</code> (Gazebo), <code>ona_config_bpsim.json</code> , or yours (K.5)
<code>--serial gw1=COM3 (x3)</code>	a real gateway board on a serial port
<code>--udp HOST:PORT</code>	gateway lines over UDP (robots in Gazebo, the zone simulator); 0.0.0.0:47100 by default in the scripts
<code>--stdin / --exec CMD</code>	gateway lines from a pipe (bpsim) or from a command
<code>--command-post URL</code>	the dashboard (default from the config, else <code>http://localhost:3000</code>)
<code>--db FILE · --fresh</code>	queue and audit file · start with an empty queue
<code>--lte-outage 60-150,300-320</code>	simulated LTE outages (seconds after start)
<code>--no-lte · --no-sat</code>	satellite only · no satellite
<code>--strict</code>	never show unconfirmed (1/3) records
<code>--quiet</code>	less output
zone simulator option	meaning
<code>--speed S · --minutes M · --seed N</code>	simulated seconds per real second (2 in the scripts) · duration · random seed
<code>--robot executor writer</code>	which robot the gateways follow
<code>--briefing-wait S · --no-wait</code>	how long the Executor waits for a briefing (120 s) · go at once

zone simulator option	meaning
--liar gw3	that gateway re-signs changed records
--forge gw2@60	that gateway reports a forged victim at 60 s
--corrupt gw1 · --gw-down gw2@60-140	damaged frames from a gateway · a gateway off for a while
--kill 6@90	beacon 6 destroyed at 90 s
--slam-slip 150:2.5,-1.5	the robot's own map jumps by (2.5, -1.5) m at 150 s
--mission FILE --world FILE	a recorded mission and its Gazebo world instead of the built-in building

H. The beacon network (LMB2)

`bpsim` runs the real beacon firmware logic for every beacon on a simulated 868 MHz channel (walls, rubble, collisions, an attacker) and checks 13 properties at the end. `bptool` builds, signs and decodes single frames.

H.1 Run the network simulator

VM terminal	<code>cd ~/living_map/beacon_net/host</code> the simulator folder
VM terminal	<code>./bpsim > /dev/null</code> 60 simulated minutes, 14 beacons, in a few seconds. The timeline and the PASS/FAIL checks print on the screen; <code>> /dev/null</code> hides the JSON lines meant for a gateway bridge.
VM terminal	<code>./bpsim --beacons 30 --minutes 90 --rubble 2.5 > /dev/null</code> a bigger, harsher network
VM terminal	<code>./bpsim --kill 3@20 --gw-late 40 > /dev/null</code> destroy beacon 3 at minute 20, switch the gateway on at minute 40 (it must pull everything)
VM terminal	<code>./bpsim --sf 12 --minutes 120 > /dev/null</code> long range, slow radio (SF12)
VM terminal	<code>./bpsim --speed 20 head -5</code> what the gateway prints: JSON lines
VM terminal	<code>./bpsim --gateways 3 --speed 20 head -5</code> three gateways: one JSON line per gateway (" <code>gw</code> ": " <code>gw1</code> ", ...), what the ONA reads (G.2)
Windows	<code>cd /d C:\tsyp\living_map\beacon_net\windows</code> Windows: the prebuilt .exe folder (in cmd)
Windows	<code>bpsim.exe --beacons 30 --rubble 2.5 > nul</code> the same simulator on Windows (<code>run_simulation.bat</code> = no options)

bpsim option	meaning (default)
--beacons N	beacons in the chain (14, max 38)
--minutes M	simulated duration (60)
--speed S	0 = as fast as possible, 1 = real time, 20 = 20× (0)
--sf 7..12	LoRa spreading factor (9)
--round S	gossip round in seconds (0 = derived from SF: 96 s at SF9)
--digest S	gateway pull period, 0 = off (90)
--kill ID@MIN	destroy beacon ID at minute MIN (6@25; 0@0 = none)
--gateways N	number of gateways around the building (1); 3 for the ONA

bpsim option	meaning (default)
--no-attacker	no forged, tampered or replayed frames
--no-seed	do not prime new beacons with the route (to see why seeding matters)
--gw-late MIN	gateway powered on late
--rubble DB	extra attenuation per metre of debris (1.2)
--seed N · --quiet	random seed (7) · no timeline

H.2 Tests and stress matrix

VM terminal	make -C ~/living_map/beacon_net/host build bpsim and bptool
VM terminal	make -C ~/living_map/beacon_net/host test firmware compiled for the PC + C/Python/firmware cross-checks + one simulation
VM terminal	make -C ~/living_map/beacon_net/host matrix the 42-scenario stress matrix (about 20 s): must end with 42/42 PASS
VM terminal	make -C ~/living_map/beacon_net/host windows rebuild the Windows .exe files (needs the MinGW cross-compiler: <code>sudo apt install -y mingw-w64</code>)
VM terminal	make -C ~/living_map/beacon_net/host clean delete the built files
VM or Windows	python3 ~/living_map/beacon_net/python/tests/test_crosscheck.py only the Python ⇌ C checks (Windows: run_tests.bat)

H.3 Work with single frames (bptool)

The demo key "LivingMap-DEMO!!" in hex is 4c6976696e674d61702d44454d4f2121; the network id is 42 (0x2A). On Windows use bptool.exe in the windows folder (example_frame.bat runs the first line). In ~/living_map/beacon_net/host:

VM terminal	./bptool example the reference VICTIM record from the brief, as JSON and as the 44-byte frame
VM terminal	./bptool fromjson 4c6976696e674d61702d44454d4f2121 42 ' { "beacon_id": 7, "kind": "VICTIM", "gps": { "lat": 34.4312, "lon": 8.7845, "err_m": 2.1 }, "severity": 15, "next": { "id": 6, "dist_m": 8.5, "ttl_s": 7200 } }' 1758500000 any record as JSON → signed frame (the last number is the time stamp; leave it out for "now")
VM terminal	./bptool decode 4c6976696e674d61702d44454d4f2121 42 212a05000f00050001000003e038f015579411060600ffbc02002300d83bb16ac021e01037f852f084ff7484 verify and decode a frame, e.g. one copied from a mission file (wrong key or one changed byte → rejected)
VM terminal	./bptool airtime 44 9 125000 5 time on air of a 44-byte frame at SF9, 125 kHz, CR 4/5: 288 ms
VM terminal	./bptool digest 4c6976696e674d61702d44454d4f2121 42 900 0 5:1 7:2 a signed pull digest from node 900 listing records 5 (seq 1) and 7 (seq 2)
VM terminal	./bptool hmac 4c6976696e674d61702d44454d4f2121 00ff raw HMAC-SHA256 of a hex message (test vectors)

I. One gateway, no ONA: other ways into the dashboard

Without the ONA a single gateway (or a bridge playing one) posts straight to the dashboard. Quicker to set up, no vote and no robot positions. Start the dashboard first (part D), then one of these.

source	section	shows
simulated beacon network (bpsim)	I.1	how the network behaves: gossip, lost beacons, aging, spoofing
the Writer in Gazebo (beacon_record_node)	I.2	the beacons the Writer drops, as it drops them
the Writer, original contract (ros_bridge.py)	I.3	the same, older format (no LMB2 fields)
a real gateway board on USB	I.4	real beacons over the air

I.1 Simulated network to dashboard

Windows	<code>C:\tsyp\living_map\beacon_net\windows\simulate_to_dashboard.bat</code> simplest: bpsim at 20x real time straight into the Windows dashboard; beacons appear over a few minutes
Windows	<code>py -3 ..\python\beaconnet\gateway_bridge.py --sim="--speed 20 --beacons 30 --rubble 2.5" --server http://localhost:3000</code> from cmd in beacon_net\windows, with any bpsim options
VM terminal	<code>cd ~/living_map/beacon_net/host && ./bpsim --speed 20 python3 . ./python/beaconnet/gateway_bridge.py --stdin --server http://10.0.2.2:3000</code> the simulator in the VM, dashboard on Windows
VM terminal	<code>cd ~/living_map/beacon_net/host && ./bpsim --speed 20 python3 . ./python/beaconnet/gateway_bridge.py --stdin --dry-run</code> no dashboard: print what would be posted

I.2 The Writer (Gazebo) to dashboard, LMB2 records

Run it in a new terminal while `run_explorer.sh` runs (without `ona`). It turns every dropped beacon into the same LMB2 record as the mission file and posts it, acting as a perfect radio network.

VM terminal	<code>python3 ~/living_map/beacon_net/ros/beacon_record_node.py --ros-args -p dashboard:=http://10.0.2.2:3000</code> dashboard on Windows
VM terminal	<code>python3 ~/living_map/beacon_net/ros/beacon_record_node.py --ros-args -p dashboard:=http://localhost:3000</code> dashboard in the VM
VM terminal	<code>ros2 topic echo /writer/beacon_record</code> the records it builds (JSON + frame hex)

parameter	default	meaning
dashboard	(empty = off)	dashboard address to post to
serial	(empty = off)	USB port of a beacon to provision (K.4)
anchor_lat, anchor_lon	36.8065, 10.1815	where the map origin is on Earth (the start point)
map_yaw_deg	0	compass direction of the map +x axis (0 = east)
key	demo	mission key: demo or 32 hex characters
net_id	42	network id (0x2A)

parameter	default	meaning
err0_m, err_per_m	0.3, 0.02	position error model
vision_confidence	0.9	confidence of camera detections
gateway_id	writer	name of the gateway chip on the dashboard

I.3 The Writer to dashboard, original contract (ros_bridge.py)

VM terminal `python3 ~/living_map/command_post/ros_bridge.py --ros-args -p server:=http://10.0.2.2:3000`
 older bridge: posts each `/writer/beacon_dropped` in the original brief format (no LMB2 fields) with a gw1 heartbeat. Parameters: `anchor_lat`, `anchor_lon`, `map_yaw_deg`, `gateway_id`, `send_trail` (false = hazards only). Use I.2 or I.3, not both.

I.4 A real gateway board to dashboard

Windows `C:\tsyp\living_map\beacon_net\windows\real_gateway_to_dashboard.bat`
 asks for the COM port (Device Manager → Ports, e.g. COM5) and the mission key (Enter = demo), sends KEY and TIME to the board, then posts everything it hears

Windows `py -3 ..\python\beaconnet\gateway_bridge.py --serial COM5 --key demo --server http://localhost:3000`
 the same by hand (from `beacon_net\windows`)

VM terminal `python3 ~/living_map/beacon_net/python/beaconnet/gateway_bridge.py --serial /dev/ttyUSB0 --key demo --server http://10.0.2.2:3000`
 in the VM (needs Devices → USB passthrough of the board and the dialout group)

VM terminal `ls /dev/ttyUSB* /dev/ttyACM*`
 which USB serial ports the VM sees

gateway_bridge.py option	meaning
<code>--serial PORT --stdin --sim "ARGS" --exec CMD</code>	where the JSON lines come from (choose one)
<code>--server URL</code>	dashboard address (default <code>http://localhost:3000</code>)
<code>--key HEX demo, --net N</code>	serial only: provision the gateway board with the mission key and network id
<code>--baud N</code>	serial speed (115200)
<code>--gw NAME</code>	gateway name on the dashboard (default from the board's heartbeat, else gw1)
<code>--dry-run · --quiet</code>	print the payloads instead of posting · less output